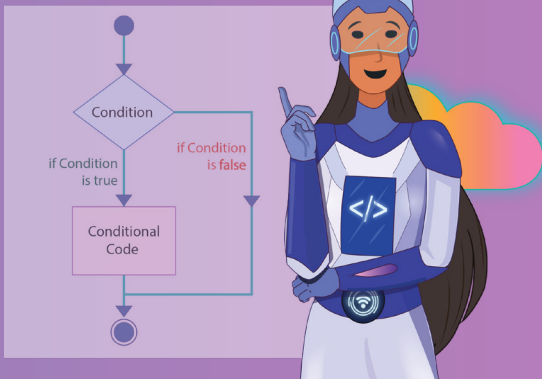


Develop Simulation



We can easily recreate a real-life situation in a virtual environment with a simulation using Javascript.



In the last few lessons, we've learnt basics of JavaScript. In this lesson, we will look at more concepts like operators and conditions in JavaScript and how to use them. But first things first, do you know what a simulation is? Let's understand.

Have you ever been inside an airplane's cockpit? There are hundreds of buttons and controls which pilots use to fly a plane. But how do pilots learn to use them before they can fly an actual plane?

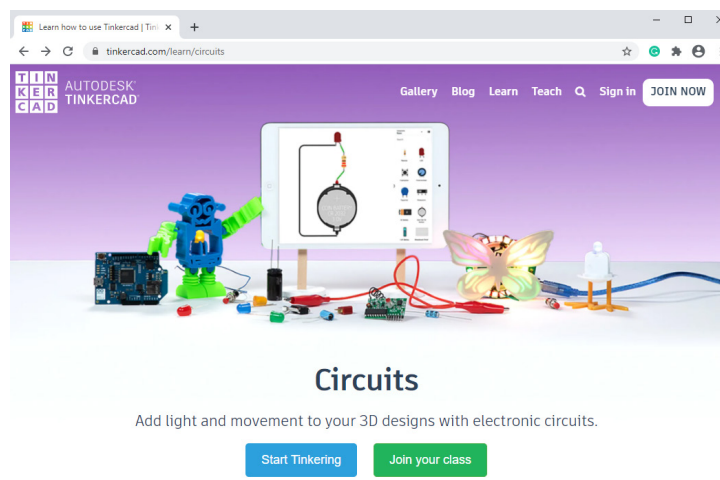
When pilots first learn to fly, they use a simulation, or an imitation of the cockpit. This is created on ground and mimics the same environment. A simulation is an approximate imitation of a physical process or system using computer applications and devices. Through a simulation application, we can experience a system without using the real thing. It provides learnings and experiences, without wasting any resources.



Flight Simulators

There are software applications to simulate the working of different machines, physical behaviours of objects, instruments, electrical and electronic circuits, etc. All simulated systems are created with programming and computers at their core.

For example, Tinkercad Circuits is a software tool where people can easily build virtual electronics circuits and simulate their results. (<https://www.tinkercad.com/learn/circuits>). These simulation applications are created using code.



We will create a simulation of a simple electronic circuit with an LED light and a switch that controls it. Before we start, let us understand some important terms that we need to know.



Recall:

Battery: A battery is a container that stores energy in the form of electric potential or voltage.

Electronic devices require different amounts of voltage to operate. Some devices require a 1.5 V battery, and some need a 9 V battery.

Voltage: Voltage is an electric force that causes electric current to flow from a point of high voltage to a low voltage one.

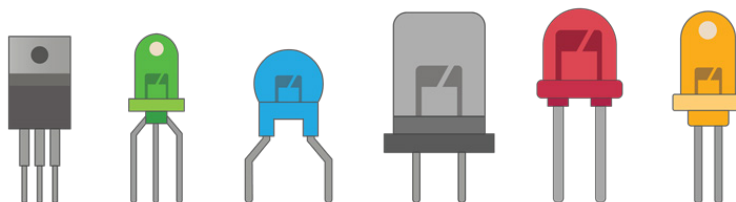


• Current

Current is a stream of charged particles. Current always flows from a positive to a negative point. Voltage pushes current through the wire.

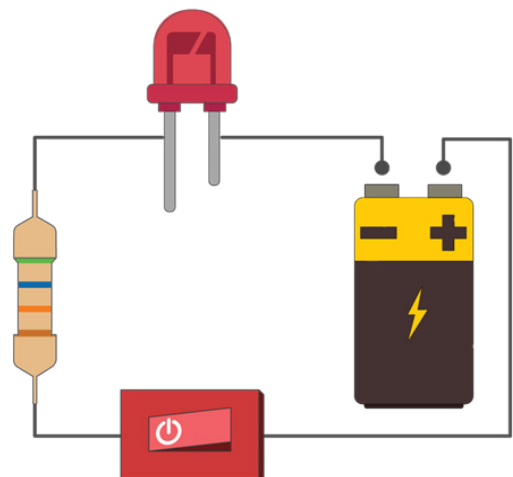
• LED

An LED is a tiny light source used in electronic circuits as an indicator, among other purposes.



• Circuit

A circuit is a closed path that allows electric current to flow from one point to another. It can include various electronic components like a battery, resistors, LEDs, etc. For example, let's look at the following circuit that contains a red LED light, a battery, a switch, and a resistor.



When the switch of a circuit is 'On', current passes through the circuit and the LED lights up. When the switch is 'Off', it disconnects the closed loop of the circuit. After this, the current cannot pass and the LED stays off.

If we try to build a circuit like this in real life without knowing the right properties of the components, it could damage some of the components and alter the desired results.

To overcome this, we can simulate a circuit which shows us the potential result. This also ensures our creation or experiment is safe and result oriented.

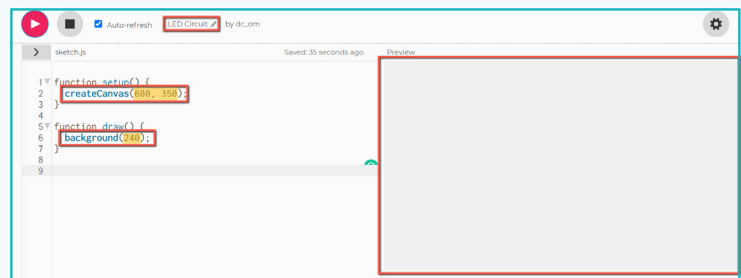
Let's use p5js to create a circuit simulation.

Activity

Create a circuit simulation using p5js

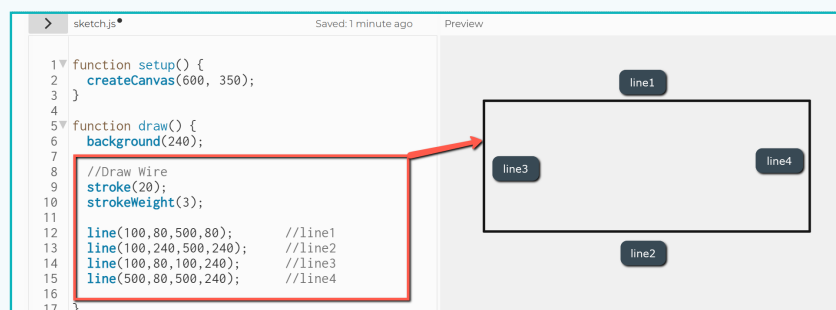
1 Initialise the project

- Open p5js editor and change the project name to LED Circuit.
- Edit the existing code to make the canvas size 600 x 350.
- Change the background colour to 240.



2 Draw wires for the circuit

- Set wire colour using stroke(20).
- Set wire thickness using strokeWeight(3).
- Use line(x1, y1, x2, y2) to draw lines for a wire loop in a rectangle.



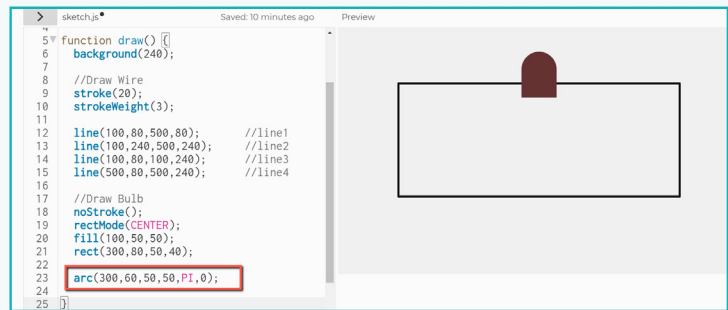
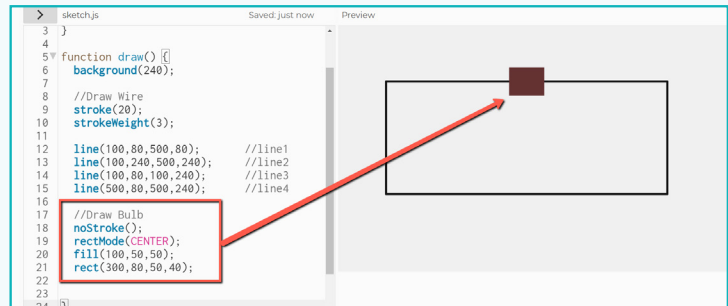
Activity

3 Draw a bulb

The shape of an LED is a half-oval. We can draw it as a rectangle with a hemisphere on top.

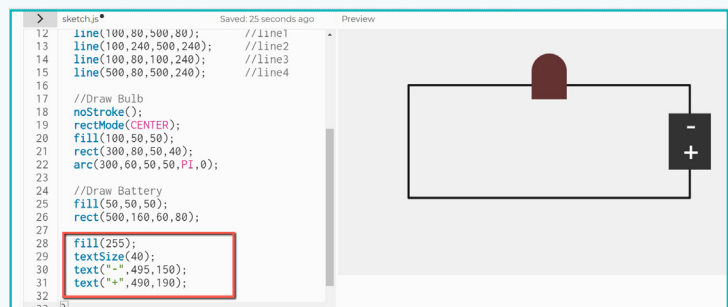
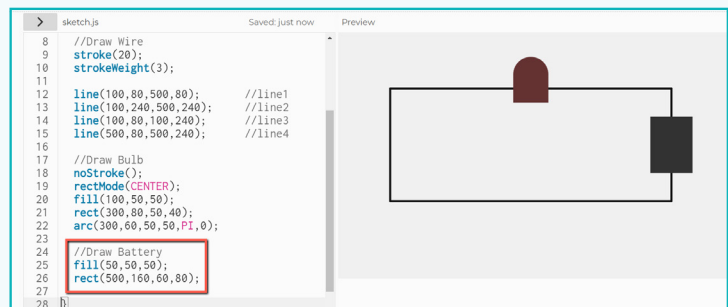


- Add `noStroke()` to draw a shape without a border.
- Add `rectMode(CENTER)` to draw a rectangle using a centre point.
- Set its colour with values R=100, G=50, B=50
- Add `rect(x, y, l, b)` to draw a rectangle centred at x=300, y=80 with length=50 and breadth=40.
- To draw the upper part of the LED bulb, draw a semicircle using `arc(x, y, width, height, start_angle, end_angle)` at x=300 and y=60.



4 Draw a battery

- Add a rectangle to represent a battery in the circuit.
- To show positive '+' and negative '-' terminals of the battery, use `text("text", x, y)` command. This lets us add text at the point specified.
- Add `fill(255)` to set the text colour as white.
- Use `textSize(40)` to set a larger text size and make it look like a battery.
- Add symbols + and - on the battery using `text("-", 495, 150)` and `text("+", 490, 180)` commands.

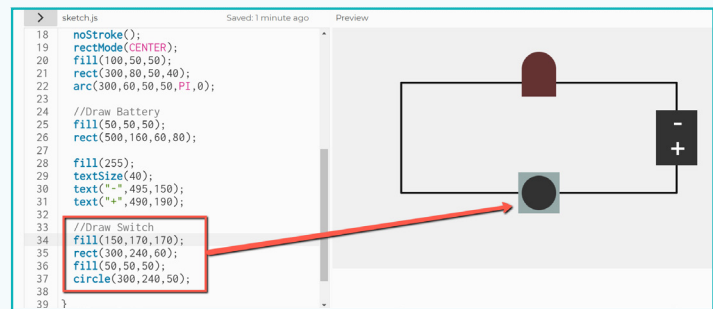


Activity

5 Draw a switch

We will create a round shaped push button on a square base for a switch. Push buttons are electrical switches that are actuated by pressing them.

- Add fill() command with colour R= 150, G=170, B=170.
- Add rect() to draw a square for the switch.
- Add fill() with colour R= 50, G=50, B=50 of the button.
- Add circle() to draw a circle on the square as a push button.

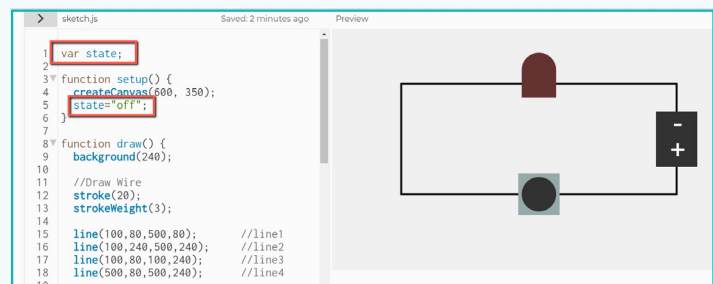


When clicked, it switches the circuit 'On'. When clicked again, it switches the circuit 'Off'.

6 Add functionality to the switch

We will use variables to add functionality to the switch. The variables will save the present state of the circuit. If the switch is on, the variable value will be 'on', otherwise it will be 'off'. This can be used to make visual changes based on switch state.

- Create a variable named 'state' at the beginning of the code.
- Set its initial value as 'off' in setup() function.



Using the status variable, we can switch the LED on and off with the if-else condition.

- If else:** A condition if-else is a code structure that executes either part 1 of the code or part 2 of the code, based on the condition fulfilled.

```

if(condition) {
  //code part 1
} else {
  //cpde part 2
}

```

A condition is placed inside brackets next to 'if'. When that condition is true, the code inside the curly brackets runs. When the condition is false, the code inside curly brackets after the 'else' block runs instead. We usually need comparison operators for adding a condition.

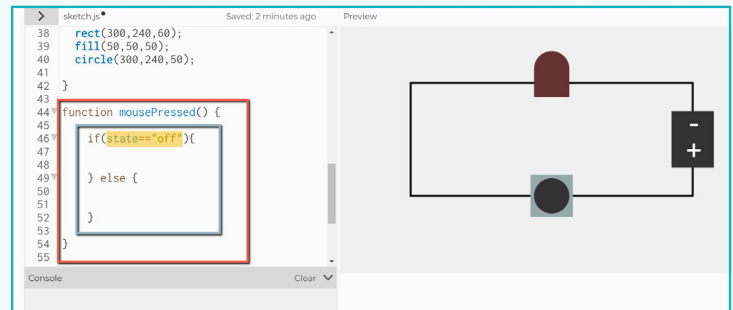
- **Comparison Operators:**
Comparison operators are used to compare values.

Operator	Meaning
=	Equal to
!=	Not equal to
>	Greater than
<	Lesser than
>=	Greater than or equal to
<=	Less than or equal to

Activity

To check when a mouse button is clicked:

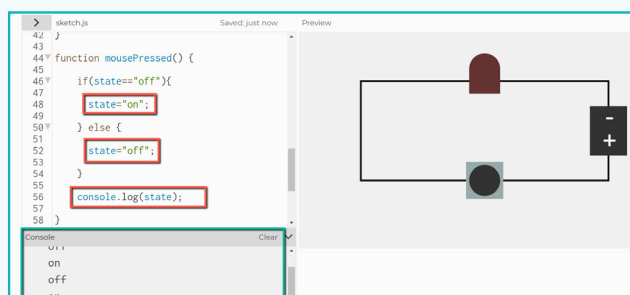
- Add default function `mousePressed()` after `draw()` to check if the event of a mouse click has occurred.
- Add if-else condition to toggle the 'state' variable based on the value.
- Add a condition to check if the value in the state variable is "off" with **`(state=="off")`**.



Note

To check if two entities are equal, '=' sign is used twice: **`"=="`**.

- To toggle the state value, add code `state="on"`; in the first block if the condition **`(state=="off")`** is true.
- If the condition is not true, i.e. if value inside state variable is already "on", the else block will run. Add code `state="off"`, in the else block.
- Add `console.log(state)` command to check the value of the state variable in the console.

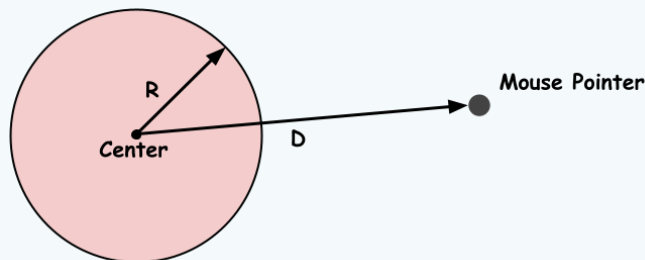


Activity

When we click on the canvas, the state variable value will toggle between on and off. We need this to happen only when we click on the circle button.

To check if the mouse pointer is inside the circle, we need to find the distance between the centre of the circle and the mouse pointer. We will use `dist()` command.

- `dist (x1, y1, x2, y2)` command displays the distance between points (x1, y1) and (x2, y2).



If the distance 'D' is greater than the radius of the circle, the mouse pointer will be outside the circle. If the distance 'D' is less than the radius of the circle, the mouse pointer will be inside the circle.

- Create a local variable 'd' inside `mousePressed()` function to store the distance between the centre of the circle (300,240) and the mouse pointer.
- Assign variable 'd' a value of `dist(300,240,mouseX,mouseY)`;

```

43
44 function mousePressed() {
45   var d=dist(300,240,mouseX,mouseY);
46
47
48   if(state=="off"){
49     state="on";
50   } else {
51     state="off";
52   }
53
54   console.log(state);
55 }
56
57
58
59
  
```

- Add if condition and insert the if-else code for the state inside the if condition.
- Add a condition in the if statement as (`d < 25`). The circular button's diameter is 50, so its radius is 25.

```

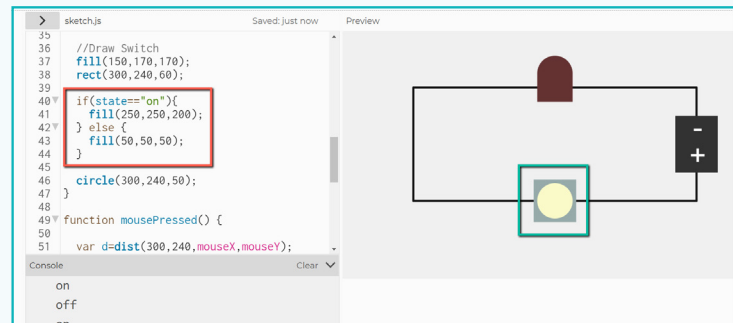
42 }
43
44 function mousePressed() {
45   var d=dist(300,240,mouseX,mouseY);
46
47   if(d<25){
48     if(state=="off"){
49       state="on";
50     } else {
51       state="off";
52     }
53   }
54   console.log(state);
55 }
56
57
58
59
  
```

Console: off, on, off

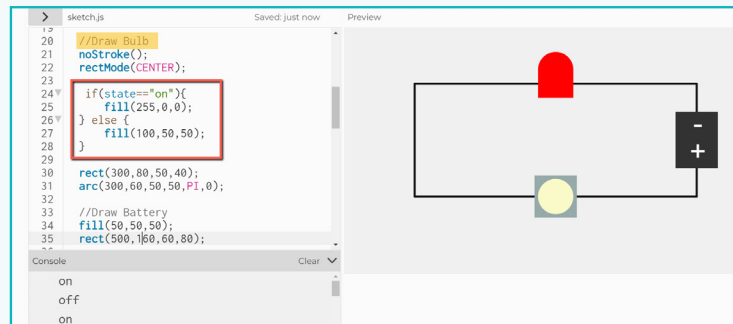
Activity

It will look great if we visually depict the button switching on and off. In draw() function, add if-else condition to change the button colour based on the value in the state variable.

- m. Add if-else condition in draw() to set the colour of the switch button i.e. the circle.
- n. Add condition if (state=="on") to check the state variable value.
- o. If the condition is true i.e. state=="on", add fill(250,250,200) in the if section.
- p. Add existing fill(50,50,50) command to the else section if state=="off".



Use similar steps to add code to change the LED colour. When the switch is on, it should turn bright red. When the switch is off, it should become dark again. Use If-else condition to code this.



When we click on the button to turn the switch 'on', the LED light turns on and becomes red. If we click the button again, it turns off.

Exercise

Fill in the blanks

1 To draw a semi-circle, _____ command can be used.

a. Circle

☐

b. Ellipse

☐

c. Arc

☐

d. Line

☐

Exercise

Fill in the blanks

2 _____ is a comparison operator.

a. > ☐

b. < ☐

c. = ☐

d. All of the Above ☐

3 _____ is used to run only one of the two or more code blocks based on the condition.

a. If ☐

b. While ☐

c. If-Else ☐

d. All of the Above ☐

Light Intensity Control

When a high voltage is applied to the same circuit, a higher current will flow through it. If it is above a certain limit, it may damage the LED. We can limit the current flowing through by increasing the resistance of the circuit. We do this with a component called a resistor.

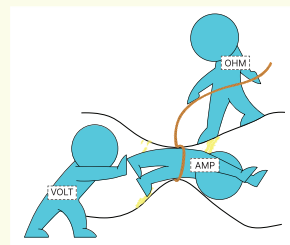
Each component in a circuit consumes some energy, which resists the flow of current. The voltage from the battery and resistance from the circuit components determine the amount of current that flows.



Recall:

Resistance: Resistance is the quality of a material which limits the current flowing through it. Higher the resistance, lesser the flow of current.

Resistor: A resistor is a component with a specific amount of resistance. In electrical and electronic circuits, resistors are used to control the flow of electric current.

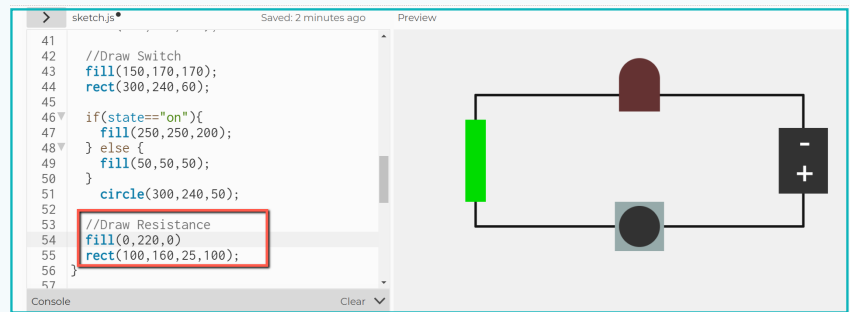


Activity

In this part of the lesson, we will add a resistor to the circuit.

7 Add a resistor

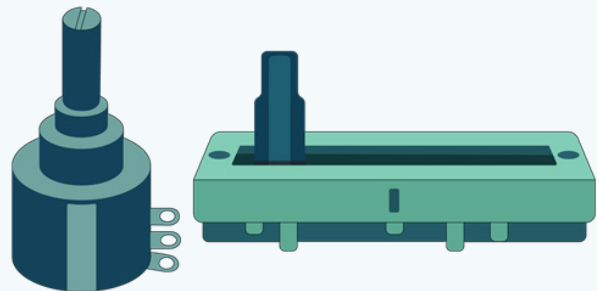
- Add fill() command with R=0 , G=220 , B=0.
- Add rect() to draw a rectangle at x=100, y=160. This indicates a resistor in the circuit.
- Set length =25, height=100 since resistors are usually taller and not wider.



The LED will be safe and not get damaged if a resistor with a compatible resistance value is added to the circuit.

To change the intensity (glow) of the LED, we will add a variable resistor.

- Variable Resistance:** A resistor whose resistance value can be adjusted. We do this with a sliding or rotating contact or knob.



Let us convert the existing resistor from the circuit into a variable resistor.

Activity

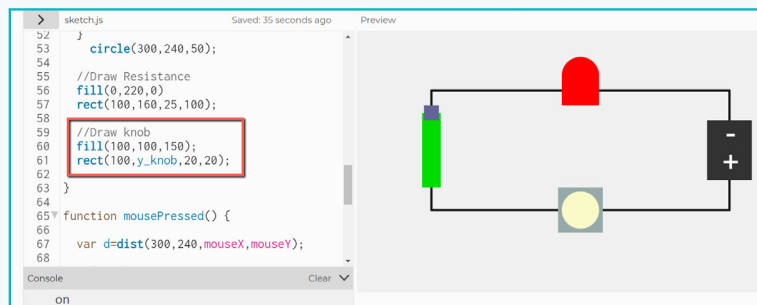
8 Add a knob to the variable resistor

- Create a variable called 'y_knob' at the top of code. The y position value of the knob will be saved in the variable. It will be later mapped to bulb color to depict the change in the resistance value.
- Set the initial value of y_knob = 110 in setup() function to place it at the initial position.
- Add fill() to set the knob colour as R=100 , G=100 ,B=150.
- Add rect() to draw the knob with position x=100, y=y_knob, with sides=20.

```

> sketch.js Saved: 2 minutes ago
1 var state;
2 var y_knob;
3
4 function setup() {
5   createCanvas(600, 350);
6   state="off";
7   y_knob=110;
8 }
9
10 function draw() {
11   background(240);
12
13   //Draw Wire
14   stroke(20);
15   strokeWeight(3);
16

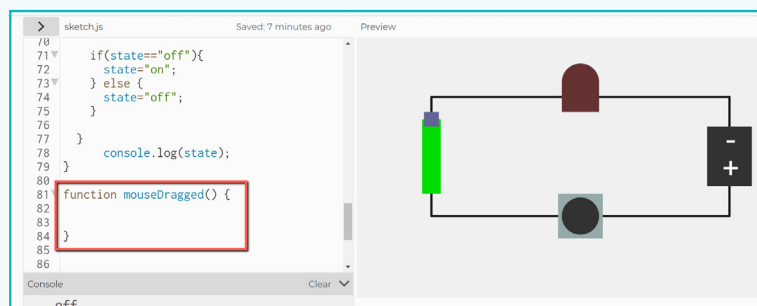
```



9 Add functionality to the variable resistor knob

To change the resistance, we need to move the knob.

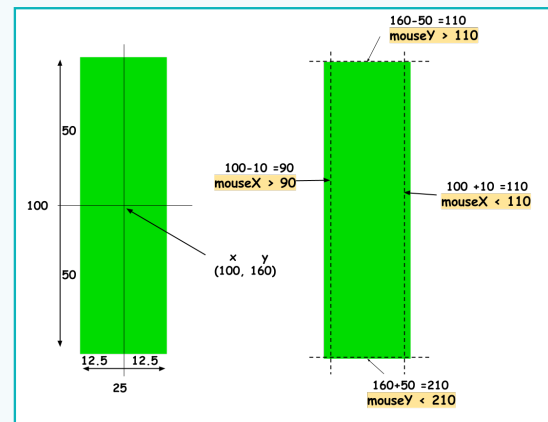
- Add mouseDragged() default function in the code.



Activity

Now, we need to detect if the mouse pointer is on the resistor when it is dragged. We will use if condition and check if the pointer is within all the edges of the resistor.

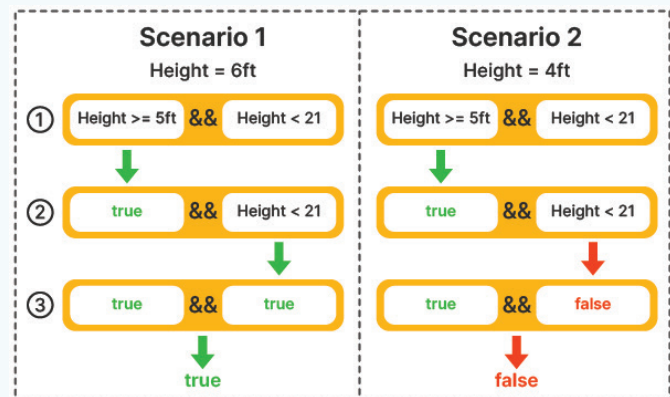
- b. First, let's find out where the right, left, bottom and top edges of the resistor are. It is located at $x=100$ and $y=160$, with its width = 25 and height = 100.



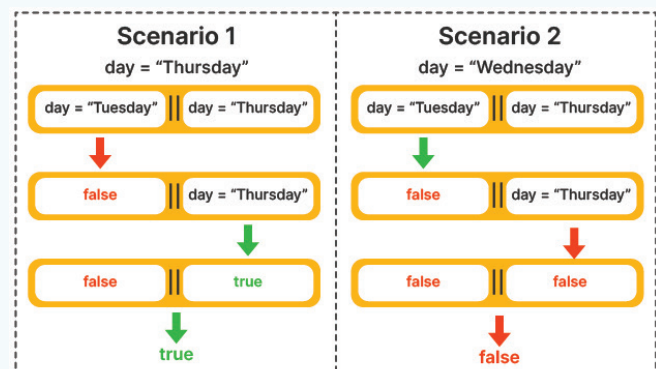
When all four conditions hold true, the mouse pointer will be on the resistor. We can use logical operators to check for multiple conditions in if condition.

Logical Operators or Boolean Operators: We use logic while coding and perform logical operations using 'OR', 'AND' and 'NOT' logical operators. These operators are used to combine two conditions and get a single result.

- c. **AND operator (&&):** If both conditions combined with && are true, only then the output is 'True'. If either one or both conditions are false, the output is false.

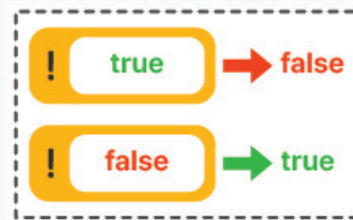


- d. **OR operator (||):** If one or more of the conditions combined with || are true, then the output is true. Only if all the conditions are false, the output is false.



Activity

e. NOT operator (!): NOT operator is used to reverse the value a variable return.



In our case, all four conditions must be true, so we need to use AND (&&) operator.

```
mouseX > 90
mouseX < 110
mouseY > 110
mouseY < 210
```

- f. Add if condition in mouseDragged() function. Combine the 4 conditions above and add them using && operator.
- g. We want to drag the knob with the mouse pointer. Inside the if block, the position of y knob = mouseY.

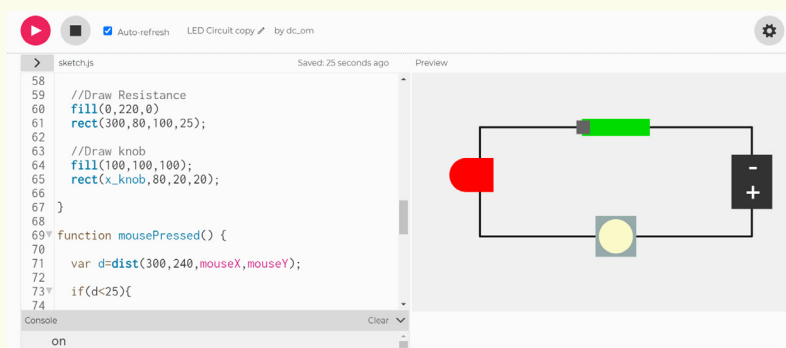
```

> sketch.js Saved: 15 seconds ago
79   }
80   }
81   }
82   console.log(state);
83   }
84   }
85   function mouseDragged() {
86   }
87   if(mouseX>90 && mouseX<110 && mouseY>110 && mouseY<210){
88   }
89   y_knob = mouseY;
90   }
91   }
92   }
93   }
    
```

Now, we can move the knob over to the resistor.

Note

We can change the positions or sequence of the components in a physical single loop circuit without any change in its working. In case of the simulation, we will have to edit the code to change positioning and conditions for correct functionality.



Exercise

State whether True or False

- 4 If AND operator is used, then the output is true only when all the expressions joined by AND operator are true.

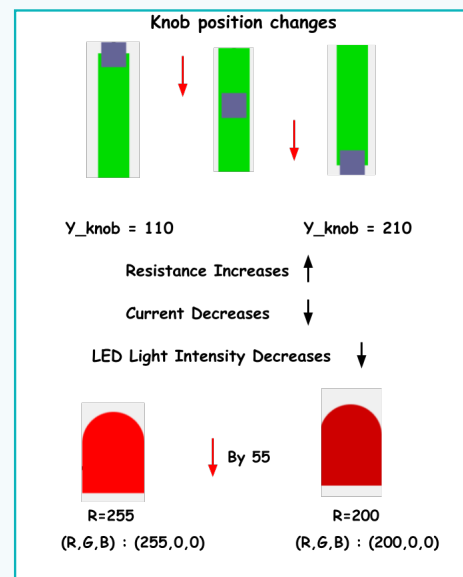
- 5 If one of the expressions joined by OR operator is true, the output will be true.

- 6 To run the code in the event of mouse dragged with button pressed, mouseClicked() default function is used.

Activity

10 Add functionality to the variable resistor

- The value of the resistor will increase after we move the knob. The circuit resistance will increase, reducing the current flowing through the circuit. This, in turn, reduces the LED light intensity.



9. Develop Simulation

The change in `y_knob` results in a change in resistance which changes the LED colour. We can link `y_knob` and colour values with the `map` command.

map(num, start1, stop1, start2, stop2):

The `map` command maps the value in the 'num' variable from range 1 to range 2. For example, If the value in num variable is between the first range that is from 0 to 10 and the second range is from 0 to Pi, map function will give the output as $\text{Pi}/2$, if `num = 5`.

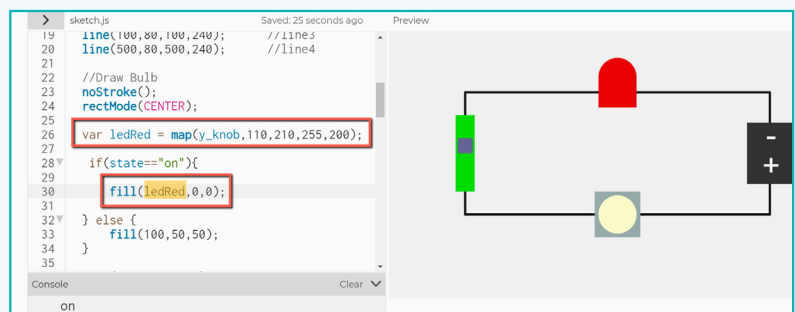
Num	map(num, 0, 10, 0, Pi)
0	0
2	$2 \cdot \text{Pi}/10$
5	$5 \cdot \text{Pi}/10 = \text{Pi}/2$
10	Pi

Activity

- a. We want to link the change in `y_knob` of range 110 - 210 to the Red colour value of LED from 255 to 200.

Y_knob	map(y_knob, 110, 210, 255, 200)
110	255
210	200

- b. Create a local variable called `ledRed` and assign values using `map()` as `map(y_knob, 110, 210, 255, 200)`;
- c. In the fill command for the LED bulb, replace 255 by `ledRed` variable. This will reduce the light's brightness depending on the value in `y_knob`.



We can change the LED light's intensity using variable resistance. Through this process, we have created a simulation of a simple electronic circuit with a battery, switch, an LED, and a variable resistor.

In this lesson we learnt how to create a simulation of a circuit using a code. Simulations enable us to create safe and result oriented experiments without any expensive equipment.

Exercise

Answer the questions in one line

7 A variable price = 10. What will be the output of the map command map(price, 0, 10, 0, 100)?

8 A variable price = 5. What will be the output of the map command map(price, 0, 10, 0, 100)?

Tech Flash

- **If-Else Condition** : Used to run either part 1 or part 2 of a code, based on the condition.
- **Comparison Operator** : Operators like >, < or ==, that are used to compare values.
- **Logical Operator** : Used to perform logical operations on two or more expressions.
- **AND Operator** : When AND operator is used, the output will be true only if all the conditions combined with AND (&&) are true. If either one or all conditions are false, the output is false.
- **OR Operator** : When OR operator is used, the output will be true if one or more of the conditions combined with OR (||) are true. The output is false only if all conditions are false.
- **NOT Operator:** : Used to reverse whatever value a variable or comparator returns.